

Teoría de PSyD

Daniel Báscones García

9 de septiembre de 2026

Índice

1. Del código de alto nivel a la memoria física	2
1.1. La estructura del hardware - dispositivos mapeados en memoria	2
1.2. El acceso a memoria - punteros y direcciones	3
1.3. Las operaciones bit a bit - manipulación de registros	4
1.4. Definiciones vs variables - el espacio utilizado	5
2. Estructuras y definiciones de hardware	7
2.1. La palabra clave <code>volatile</code>	7
2.2. Abstracción de Hardware con <code>struct</code>	7
2.3. Utilización de <code>union</code> y campos de bits	8
2.4. Palabra clave <code>static</code>	9
2.4.1. Funciones <code>static inline</code>	9
3. El enlazado de programas e inicialización	10
3.1. El <i>linker script</i>	10
3.2. Código de inicio y salto a <code>main</code>	11
3.3. La tabla de vectores	12
3.3.1. El modificador <i>weak</i>	13
4. Interrupciones, sincronización y <i>callbacks</i>	14
4.1. Barreras de memoria	14
4.2. Punteros a función y Callbacks	15

1. Del código de alto nivel a la memoria física

Los lenguajes de alto nivel son cada vez más abstractos. En un origen, la programación se hacía directamente en ensamblador, atacando directamente al conjunto de instrucciones del propio procesador. Con la creación y avance de los compiladores, estos se sustituyeron por lenguajes algo más abstractos como COBOL, Pascal, o en última instancia C, incluyendo conceptos como funciones o tipos, e incluso multitarea. Posteriormente, se introdujeron lenguajes más complejos como C++, que añade clases, orientación a objetos, plantillas, y multitud de comodidades en versiones más recientes como programación funcional, pero aún con posibilidades cercanas al hardware. Hoy en día, la mayor parte de los programadores trabajan en lenguajes de aún más alto nivel, como Java o Python, que ni siquiera se ejecutan en su mayor parte directamente sobre hardware, sino sobre máquina virtual, lo cual permite estilos de programación muy flexibles donde el concepto de tipos o funciones se diluye.

Esta evolución la permite, en parte, el *stack* tecnológico que vive debajo: Un procesador complejo, sobre el que se ejecuta un sistema operativo, que a su vez tiene un intérprete instalado al que ofrece interfaces de control, sobre el cual finalmente se ejecuta el lenguaje de nuestra elección.

Sin embargo, sigue habiendo nichos donde tener toda esta pila de herramientas no es posible: uno de ellos son los sistemas empotrados.

Un sistema empotrado, aunque su definición sea en ocasiones difusa, suele tener capacidades muy reducidas, y estar diseñado para funciones muy específicas. Dada la simplicidad de su diseño, los procesadores que incluyen no soportarían (o serían demasiado lentos) un sistema con la complejidad como para ejecutar todos estos lenguajes modernos. Por tanto, aún se siguen utilizando lenguajes más simples y con características más cercanas al hardware. En concreto, el lenguaje rey en este ámbito sigue siendo C. Aún y con todo, C sigue siendo un lenguaje de alto nivel, y conviene entender cómo nos va a permitir trabajar directamente con el *hardware*.

1.1. La estructura del hardware - dispositivos mapeados en memoria

Un procesador, por sí solo, no es más que una máquina de ejecutar instrucciones. Según el conjunto de instrucciones con que diseñemos nuestro procesador, le daremos más o menos capacidades. Aún así, muchas de las capacidades no residen en el propio núcleo en sí, sino en periféricos asociados que liberan de trabajo al *core* ejecutor de tareas.

En las arquitecturas de microcontroladores modernas (como la familia ARM Cortex-M), el procesador utiliza, para comunicarse con dichos periféricos, un esquema denominado *Memory-Mapped I/O* (MMIO). Así, para comunicarse con los periféricos del procesador, no se utilizan instrucciones específicas. En su lugar, todos los periféricos comparten el mismo espacio de direccionamiento de 32 bits (4GB de direcciones) junto con la memoria de programa (Flash) y la memoria de datos (RAM).

- **Memoria Flash** (0x08000000): Almacena el código compilado, constantes, e información persistente. Es memoria no volátil, que se mantiene tras un apagado o reinicio del sistema.
- **Memoria SRAM** (0x20000000): Almacena las variables globales, la pila (*stack*) y el montón (*heap*). Es memoria volátil de acceso rápido, que se reinicia en cada inicio del sistema.

- **Periféricos** (0x40000000 - 0x50000000 y otros): Colección de registros físicos mapeados directamente en direcciones de memoria fijas.

Desde la perspectiva del procesador, escribir en la dirección 0x40020000 no difiere sintácticamente de escribir en una variable almacenada en la RAM. Sin embargo, la circuitería del bus redirige esa operación al controlador del periférico correspondiente (por ejemplo, puerto de entrada/salida de propósito general), provocando un cambio de estado ajeno al procesador, como por ejemplo el encendido de un LED.

1.2. El acceso a memoria - punteros y direcciones

Para interactuar con las direcciones físicas descritas en el modelo MMIO desde el lenguaje C, se emplean punteros. En C, debemos trabajar manualmente con valores numéricos (representando dichas direcciones) para conseguir acceder a las mismas. Un puntero se describe de la siguiente manera:

```
1 #include <stdint.h>
2
3 uint32_t * puntero;
```

NOTA: Es fundamental utilizar tipos de datos con ancho de bits garantizado mediante la librería de la cabecera estándar `<stdint.h>` (como `uint32_t`, `uint16_t`, `uint8_t`), evitando tipos ambiguos como `int` o `long` cuyo tamaño puede variar según arquitectura y compilador.

Un puntero no es más que una dirección de memoria. En este caso, `uint32_t *` representa “dirección de memoria donde hay almacenado un `uint32_t`”. Si queremos cambiar ese valor, guardado en dicha dirección de memoria, haremos:

```
1 *puntero = 20;
```

Y, si queremos cambiar la dirección de memoria a la que apuntamos, haremos:

```
1 puntero = (uint32_t *) 0x40020000U;
```

Así, conociendo la dirección de memoria a través de la cual podemos acceder a un periférico (por ejemplo para encender un LED), podemos escribir en ese preciso lugar de la siguiente manera:

```
1 #include <stdint.h>
2
3 // Macro para poder cambiar fácilmente la dirección
4 #define DIRECCION_OBJETIVO (0x40020000U)
5
6 void encender_led(void) {
7     // 1) creación de nuestro puntero (indicamos el tipo del mismo)
8     uint32_t * p_led = (uint32_t *) GPIOA_MODER_ADDR;
9
10    // 2) Escribimos sobre esa dirección de memoria
11    *p_led = 0x00000001;
12 }
```

En la práctica, se suelen combinar ambas operaciones en una única macro para facilitar la escritura de código:

```

1 #define REGISTRO_LED (*(volatile uint32_t *) 0x40020000U)
2 // Ahora podemos utilizar la macro como si fuera una variable
3 REGISTRO_LED = 0x00000001;

```

Si aún quedan dudas, consulta la Figura 1 para verlo en forma de diagrama.

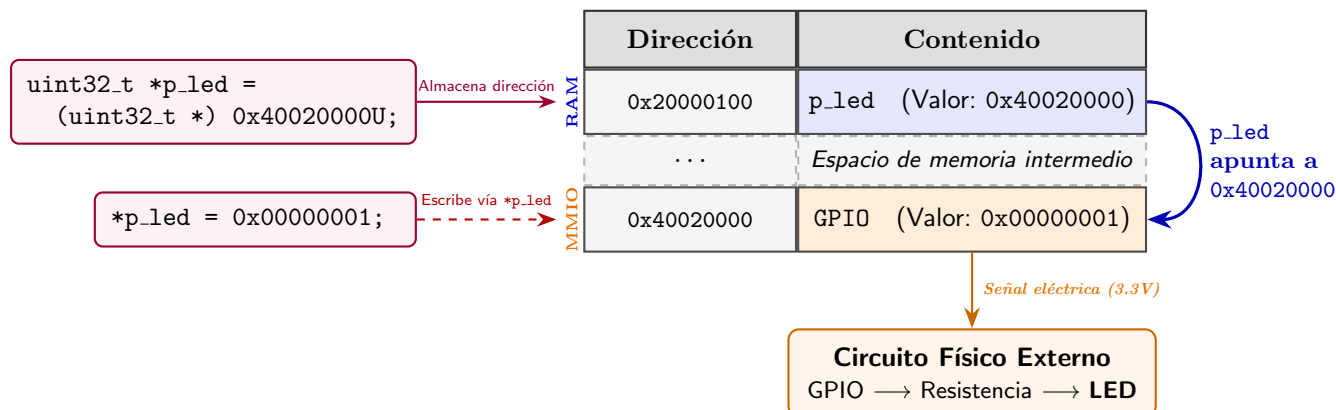


Figura 1: Esquema de las dos direcciones de memoria involucradas con un puntero: la dirección a la que apunta, y el valor apuntado.

NOTA: Si hablamos de punteros, no podemos dejar de lado (aunque no se utilice habitualmente) el operador `&`. Este operador sirve para darnos la *dirección* de una variable. Así, podremos establecer punteros a direcciones concretas donde tenemos almacenados datos:

```

1 uint32_t    a;
2 uint32_t* ptr;
3 ptr = &a; //guardamos dirección de 'a' en el puntero
4 *ptr = 20; //modificamos 'a' directamente

```

1.3. Las operaciones bit a bit - manipulación de registros

Un registro de un periférico de 32 bits suele alojar múltiples configuraciones independientes; cada bit o grupo de bits controla una función específica (por ejemplo, velocidad, dirección de un pin o activación de interrupciones). Por ello, escribir directamente un valor entero (como `REGISTRO_LED = 0x00000001`) suele ser peligroso, ya que sobrescribe y destruye la configuración de los 30 bits restantes.

Para modificar únicamente los bits de interés sin alterar los demás, se utiliza el patrón *Read-Modify-Write* (Lectura-Modificación-Escritura) mediante operadores bit a bit (*bitwise*):

■ Establecer bits a '1':

```

1 //ponemos el bit '0' a '1', resto se mantienen
2 REGISTRO_LED |= 0x00000001;
3 //ponemos todos los bits a '1' salvo el bit '0'
4 //que se mantiene
5 REGISTRO_LED |= 0xFFFFF000;

```

■ Borrar bits a '0':

```

1 //ponemos el bit '0' a '0', resto se mantienen
2 REGISTRO_LED &= ~0x00000001;
3 //borramos todos los bits salvo el bit '0'
4 //que se mantiene
5 REGISTRO_LED &= 0x00000001;

```

■ Alternar estado:

```

1 //cambiamos el bit '0' (de '1' a '0' o de '0' a '1')
2 //resto se mantienen
3 REGISTRO_LED ^= 0x00000001;
4 //cambiamos todos los bits excepto el '0'
5 REGISTRO_LED ^= 0xFFFFF0;

```

■ Leer bits individuales:

```

1 //leemos el bit '0'
2 uint32_t valor = REGISTRO_LED & 0x00000001;
3 //leemos el bit '16' y lo movemos a la derecha
4 uint32_t valor = (REGISTRO_LED & 0x00010000) >> 16;

```

Hay que tener siempre en cuenta que los bits en los que haya que escribir, dependerán del funcionamiento del periférico mapeado en memoria. También, en ocasiones, solo leer el valor desencadena funciones del periférico. Por tanto será muy importante leer su documentación. Imaginemos un registro mapeado en memoria donde hay que modificar los bits 14 y 15 para escribir una configuración:

```

1 // El Pin 7 ocupa los bits [15:14] del registro GPIOA_MODER (2 bits por pin)
2 // Se requiere escribir el valor '01' (salida) en las posiciones 15 y 14.
3
4 // Borrar bits 15 y 14 usando una máscara invertida
5 GPIOA_MODER &= ~(0b11U << (7 * 2));
6 // 0bxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
7 // 0b11111111111111110011111111111111 &
8 // -----
9 // 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxx
10
11 // Poner el valor '01' en dichos bits
12 GPIOA_MODER |= (0b01U << (7 * 2));
13 // 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxx
14 // 0b00000000000000000100000000000000 |
15 // -----
16 // 0bxxxxxxxxxxxxxxxx01xxxxxxxxxxxx

```

1.4. Definiciones vs variables - el espacio utilizado

En un microcontrolador, los recursos de memoria son escasos (por ejemplo, 512KB de Flash y 1MB de RAM). Es importante conocer qué recursos consume cada elemento en C a fin de ahorrar espacio.

- **Directivas del preprocesador (#define):** No consumen memoria RAM ni Flash por sí mismas. El preprocesador realiza una sustitución textual antes de la compilación.

- **Variables globales no inicializadas (.bss):** Se ubican exclusivamente en la RAM (pues la flash no se puede modificar directamente en tiempo de ejecución). Al arrancar el programa, el código de inicio las inicializa automáticamente a cero.
- **Variables globales inicializadas (.data):** Ocupan espacio en memoria Flash (donde se guarda su valor inicial de fábrica) y en memoria RAM (donde se copian durante el arranque para permitir su modificación).
- **Constantes (const):** Si se declaran con el calificador `const`, el compilador las ubica en la sección de solo lectura de la memoria Flash (`.rodata`), evitando ocupar memoria RAM.
- **Variables locales:** Se crean dinámicamente en la pila (*stack*) ubicada en RAM durante la ejecución de las funciones y se destruyen automáticamente al salir del ámbito.

Es importante conocer que esta distribución no es estrictamente así, ya que cambios en el script de enlazado (o *linker script*) podrán modificarla. Pero en líneas generales podemos asumir que se distribuyen de esta manera.

```

1 #define CONFIG_MASK 0x00FFU           // Sustitución textual, 0 bytes consumidos
2
3 const uint32_t calib = 0xF412AB00;    // Memoria Flash (.rodata), 0 bytes en RAM
4 uint32_t contador_global;             // Memoria RAM (.bss), 4 bytes
5 uint32_t estado_inicial = 0xAA;       // Memoria RAM (.data) + copia en Flash
6
7 void proceso(void) {
8     uint8_t buffer_local[16];         // Stack (en RAM) mientras dure la función
9 }

```

2. Estructuras y definiciones de hardware

Más allá de los accesos a memoria, para gestionar todos los elementos mapeados en memoria del procesador y periféricos, existen ciertas construcciones en C que permiten facilitar el trabajo al programador de diferentes maneras. Entre ellas, podremos evitar optimizaciones no deseadas con la palabra clave `volatile`, generar `struct` que representen periféricos, acceder a nivel de bits con los campos de bits, u optimizar llamadas a funciones `static inline`.

2.1. La palabra clave `volatile`

El optimizador del compilador asume un entorno de ejecución mono-hilo clásico, donde los valores almacenados en memoria solo cambian mediante instrucciones explícitas del propio código. Bajo esta premisa, si el compilador detecta lecturas repetidas a una misma dirección de memoria dentro de un bucle, optimiza el acceso cargando dicho valor en un registro interno del procesador y reutilizándolo, evitando lecturas redundantes al bus de datos.

En sistemas empotrados, esta optimización no es correcta: los registros mapeados en memoria, o las variables compartidas con Rutinas de Servicio de Interrupción (ISR) cambian de valor de forma asíncrona debido al hardware externo.

```
1 // Direccion de un registro de estado del hardware (por ejemplo, UART RX Ready)
2 #define UART_STATUS_REG (*(uint32_t *) 0x40001000U)
3
4 void esperar_datos(void) {
5     // El compilador lee el registro UNA SOLA VEZ y genera un bucle infinito
6     // en ensamblador si el valor inicial era 0: "if (!reg) while(1);"
7     while ((UART_STATUS_REG & 0x01U) == 0) {
8         // Bucle de espera activa (Busy-wait)
9     }
10 }
```

Al calificar una variable o puntero con `volatile`, se le indica explícitamente al compilador que no optimice sus accesos y lo lea explícitamente cada vez que el código lo requiera. Así, cada lectura o escritura escrita en C se traduce explícitamente en una instrucción física de lectura o escritura en la memoria.

```
1 // Puntero a registro hardware: siempre calificado como volatile
2 #define UART_STATUS_REG (*(volatile uint32_t *) 0x40001000U)
3
4 // Variable global modificada dentro de una ISR
5 volatile uint8_t flag_interrupcion = 0;
```

2.2. Abstracción de Hardware con `struct`

Escribir macros o punteros independientes para cada uno de los decenas de registros que contiene un periférico sería, francamente, un rollo. Puesto que los registros de un periférico ocupan bloques contiguos de memoria, es habitual agruparlos dentro de un (`struct`) de C.

Al definir la estructura, el compilador asigna a cada miembro direcciones consecutivas de memoria desde la inicial. Bastará por tanto con configurar adecuadamente esta dirección inicial del `struct`, con la que especifiquen los manuales del procesador.

```

1  #include <stdint.h>
2
3  typedef struct {
4      volatile uint32_t MODER;    // Offset: 0x00 (Modo de los pines)
5      volatile uint32_t OTYPER;   // Offset: 0x04 (Tipo de salida)
6      volatile uint32_t OSPEEDR;  // Offset: 0x08 (Velocidad de salida)
7      volatile uint32_t PUPDR;    // Offset: 0x0C (Resistencias Pull-up/Pull-down)
8      volatile uint32_t IDR;       // Offset: 0x10 (Registro de entrada)
9      volatile uint32_t ODR;       // Offset: 0x14 (Registro de salida)
10     uint32_t RESERVED[2];        // Offset: 0x18 - 0x1F (Hueco reservado en
                                   hardware)
11     volatile uint32_t BSRR;      // Offset: 0x20 (Bit Set/Reset Register)
12 } GPIO_TypeDef;
13
14 // Direccion base del periférico GPIOA
15 #define GPIOA_BASE (0x40020000U)
16
17 // Creación de la variable para manejar el periférico
18 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
19
20 void ejemplo_estructura(void) {
21     // Acceso directo a cualquier registro del periférico
22     GPIOA->ODR |= (1U << 7);    // Pone a '1' el Pin 7
23 }

```

Alineación de memoria: Es imprescindible respetar los huecos de direcciones no utilizadas mediante miembros de relleno (`RESERVED`), garantizando que cada miembro de la estructura coincida exactamente con la dirección del registro físico correspondiente. También hay que tener cuidado si se utilizan miembros de tipos cortos como `uint16_t` o `uint8_t`, ya que el compilador podría introducir huecos por su propia cuenta para que variables completas de tipo `uint32_t` estén alineadas a direcciones múltiplo de cuatro. Normalmente esto no es un problema, pero conviene tenerlo en cuenta.

2.3. Utilización de `union` y campos de bits

Aunque las operaciones bit a bit con máscaras (`&`, `—`) son el estándar industrial, C permite mapear campos de bits (*bit-fields*) mediante `struct` dentro de una `union`. Esto habilita el acceso individual a un bit o grupo de bits sin requerir desplazamientos explícitos.

```

1  typedef union {
2      uint32_t raw; // Acceso al registro completo de 32 bits
3
4      struct {
5          uint32_t ENABLE : 1; // Bit 0: Activación del periférico
6          uint32_t MODE   : 2; // Bits [2:1]: Función (00, 01, 10, 11)
7          uint32_t INT_EN  : 1; // Bit 3: Habilitar interrupción
8          uint32_t RESERVED : 28; // Bits [31:4]: No utilizados
9      } bits;
10 } CR_Register_t;
11
12 void configurar_registro(void) {
13     volatile CR_Register_t *CR = (CR_Register_t *) 0x40002000U;
14
15     // Escritura directa en campos concretos sin alterar los demás
16     CR->bits.ENABLE = 1;
17     CR->bits.MODE   = 0b01;
18
19     // 0 bien modificación del registro completo en una sola operación

```



```

20     CR->raw = 0x0000000B;
21 }

```

Advertencia sobre portabilidad: La especificación del estándar C deja a criterio de la implementación del compilador la ordenación de los campos de bits (de LSB a MSB o viceversa) y el empaquetado en memoria. Además, modificar un campo de bits genera internamente una secuencia *Read-Modify-Write* no atómica, lo que puede derivar en condiciones de carrera si el registro es accedido simultáneamente por una ISR.

2.4. Palabra clave `static`

La palabra clave `static` tiene dos usos principales en C:

1. Hacer que una variable, dentro de una función, retenga el valor entre llamadas. Aunque no le daremos este uso durante el curso, conviene saberlo.
2. Hacer que una variable o función sólo sea vista dentro de la unidad de compilación a la que pertenece. Así, esas variables o funciones no pueden ser vistas por otras unidades de compilación (normalmente archivos .c), y evitamos colisiones de nombres.

2.4.1. Funciones `static inline`

A la hora de realizar operaciones frecuentemente repetidas (como por ejemplo activar o desactivar un pin de un periférico de GPIO), la aproximación tradicional ha sido la de definir macros parametrizadas mediante `#define`. De esta forma, se evita la sobrecarga de llamar a una función específica, consistente en salvar registros en la pila, además de realizar el salto y retorno.

```

1 #define TOGGLE_PIN_MACRO(port, pin) ((port)->ODR ^= (1U << (pin)))
2
3 // Evaluacion con incremento: TOGGLE_PIN_MACRO(GPIOA, i)
4 // expande a: (GPIOA)->ODR ^= (1U << (i))

```

Las macros, sin embargo, presentan algunos inconvenientes, especialmente cuando empieza a haber muchas: La trazabilidad se hace complicada, tanto en el desarrollo como en la depuración. Adicionalmente, los errores de sintaxis se hacen más comunes debido a las reglas de sustitución del preprocesador.

Como alternativa, disponemos de las funciones `static inline`. La palabra clave `inline` hace que el compilador reemplace la llamada a la función directamente por su código fuente en el punto de llamada. Por su parte, `static`, como ya decíamos, hace que la función no sea vista fuera de la unidad de compilación, así que aunque la incluyamos desde numerosos archivos .c, evitaremos errores de duplicidad al compilar.

```

1 #include <stdint.h>
2
3 static inline void gpio_toggle_pin(GPIO_TypeDef *port, uint8_t pin) {
4     // Comprobación de tipos en compilación y sin efectos secundarios
5     port->ODR ^= (1U << pin);
6 }

```

Así, se consigue la rapidez de las macros, añadiendo las comprobaciones extra del compilador y funciones de depuración estándar.

3. El enlazado de programas e inicialización

Para convertir varios archivos fuente compilados en un único ejecutable funcional, el enlazador (*linker*) asigna direcciones físicas de memoria a cada instrucción y variable. Aunque para las instrucciones nos importa un poco menos, es importante asegurar que ciertos elementos de nuestro programa acaban en direcciones de memoria concretas. Por ejemplo, el código de arranque o ciertas funciones de interrupción (disparados ambos automáticamente) tendrán que estar en ubicaciones concretas.

3.1. El *linker script*

El *linker script* (archivo con extensión `.ld`) es el documento de configuración que indica al enlazador cómo organizar la memoria del sistema. Cumple tres funciones principales: define las regiones de memoria física (utilización de Flash y RAM), ubica las diferentes secciones de código (variables/funciones) en ellas, y además permite exportar símbolos de dirección para que el propio código sepa dónde acaba ubicado.

Un mapa de memoria típico de ARM Cortex-M divide las secciones del programa en:

- **.isr_vector:** Tabla de punteros con las llamadas a funciones de inicio e interrupciones.
- **.text:** Código ejecutable e instrucciones del programa (almacenado en Flash).
- **.rodata:** Constantes y cadenas de texto de solo lectura (almacenado en Flash).
- **.data:** Variables globales e estáticas *inicializadas* con un valor distinto de cero.
- **.bss:** Variables globales e estáticas *no inicializadas* (o inicializadas a cero).

```
1 MEMORY
2 {
3     RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 256K
4     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
5 }
6
7 SECTIONS
8 {
9     /* Tabla de vectores al inicio de la Flash */
10    .isr_vector : {
11        KEEP(*(.isr_vector))
12    } >FLASH
13
14    /* Código */
15    .text : {
16        *(.text*)
17        *(.rodata*)
18        _etext = .; //símbolo exportado al final del código
19    } >FLASH
20
21    /* .rodata (constantes, strings, etc.) */
22    .rodata :
23    {
24        *(.rodata)
25        *(.rodata*)
26    } >FLASH
```

```

27
28  /* Datos inicializados, dirección de comienzo en FLASH */
29  _sidata = LOADADDR(.data);
30
31  .data :
32  {
33      _sdata = .;          /* Inicio de datos en RAM */
34      *(.data)             /* .data sections */
35      *(.data*)            /* .data* sections */
36      _edata = .;         /* Fin de datos en RAM*/
37
38  } >RAM AT> FLASH /* Se guarda en RAM en ejecución, pero está en
39  FLASH en compilación */
40
41  /* Datos sin inicializar van a RAM directamente */
42  .bss : {
43      _sbss = .;
44      *(.bss*)
45      _ebss = .;
46  } > RAM
47
48  /* Definición del tope de la pila al final de la RAM */
49  _estack = ORIGIN(RAM) + LENGTH(RAM);
50 }

```

Es importante diferenciar la **LMA** (*Load Memory Address*) frente a **VMA** (*Virtual Memory Address*). La sección `.data` reside físicamente en Flash (**LMA**) para no perder sus valores al apagar el chip, pero el procesador debe ejecutarla y modificarla desde la RAM (**VMA**). Este tipo de inicialización la hace el código de inicio.

3.2. Código de inicio y salto a `main`

Cuando el microcontrolador recibe alimentación o se resetea, el procesador no ejecuta directamente la función `main()`. En su lugar, el hardware lee la primera dirección de memoria y salta automáticamente al `Reset_Handler`, definido en el archivo de ensamblador de arranque (`startup.s` o similar).

El `Reset_Handler` realiza la inicialización básica del sistema en el siguiente orden estricto:

1. **Inicialización del Stack Pointer (SP):** Carga la dirección del puntero de pila principal con el símbolo `_estack`.
2. **Llamada a `SystemInit`:** Configura los relojes principales (*clocks*) y la memoria Flash.
3. **Copia de la sección `.data`:** Lee el bloque de inicializadores desde la Flash (`_sidata`) y los escribe en la RAM (desde `_sdata` hasta `_edata`).
4. **Limpieza de la sección `.bss`:** Rellena con ceros la región de RAM comprendida entre `_sbss` y `_ebss`.
5. **Llamada a constructores globales (`__libc_init_array`):** Prepara librerías de C/C++.
6. **Salto a `main()`:** Salta a la rutina principal de la aplicación. Si esta finaliza, entra en un bucle infinito de seguridad (`LoopForever`).

```

1  Reset_Handler:
2      ldr    r0, =_estack
3      mov    sp, r0          /* set stack pointer */
4      /* Call the clock system initialization function.*/
5      bl    SystemInit
6
7      /* Copy the data segment initializers from flash to SRAM */
8      ldr r0, =_sdata
9      ldr r1, =_edata
10     ldr r2, =_sidata
11     movs r3, #0
12     b LoopCopyDataInit
13
14     /***** Sección de copia de datos inicializados *****/
15 CopyDataInit:
16     ldr r4, [r2, r3]
17     str r4, [r0, r3]
18     adds r3, r3, #4
19
20 LoopCopyDataInit:
21     adds r4, r0, r3
22     cmp r4, r1
23     bcc CopyDataInit
24
25     /***** Sección de inicialización con ceros de .bss *****/
26     ldr r2, =_sbss
27     ldr r4, =_ebss
28     movs r3, #0
29     b LoopFillZerobss
30
31 FillZerobss:
32     str r3, [r2]
33     adds r2, r2, #4
34
35 LoopFillZerobss:
36     cmp r2, r4
37     bcc FillZerobss
38
39     /***** Inicialización de librerías *****/
40     bl __libc_init_array
41
42     /***** Salto al programa principal *****/
43     bl main
44
45 LoopForever:
46     b LoopForever

```

3.3. La tabla de vectores

La arquitectura ARM Cortex-M requiere que las direcciones de las funciones de atención a excepciones e interrupciones estén organizadas en una posición fija de memoria, conocida como **Tabla de Vectores**. Por defecto, se ubica al principio de la memoria Flash (0x08000000).

Cada entrada de la tabla es una palabra de 32 bits que contiene la dirección de memoria de la rutina correspondiente:

- **Offset 0x00:** Puntero inicial de la pila (`_estack`).

- **Offset 0x04:** Dirección del `Reset_Handler`.
- **Offset 0x08 a 0x3C:** Excepciones del núcleo Cortex (NMI, HardFault, SysTick, etc.).
- **Offset 0x40 en adelante:** Interrupciones de periféricos propios del fabricante (USART, TIM, DMA, etc.).

```

1  .section .isr_vector,"a",%progbits
2  .type g_pfnVectors, %object
3
4  g_pfnVectors:
5  .word _estack                /* 0x00: Pila inicial */
6  .word Reset_Handler         /* 0x04: Reset Handler */
7  .word NMI_Handler           /* 0x08: NMI Handler */
8  .word HardFault_Handler     /* 0x0C: HardFault Handler */
9  /* ... Excepciones del nucleo ... */
10 .word SysTick_Handler       /* 0x3C: Timer del sistema */
11
12 /* Interrupciones de perifericos externos */
13 .word WWDG_IRQHandler
14 .word USART1_IRQHandler
15 .word EXTI0_IRQHandler

```

Podemos observar que esta sección (`.isr_vector`) es la que colocaba anteriormente el *linker script* al principio exacto de la **FLASH**, para así ubicar todos estos símbolos en las posiciones adecuadas.

3.3.1. El modificador *weak*

En el archivo de arranque, todas las interrupciones se enlazan por defecto a una rutina genérica llamada `Default_Handler` (que ejecuta un bucle infinito) mediante el atributo de ensamblador `.weak` (o `__attribute__((weak))` en C).

El modificador *weak* (débil) indica al enlazador que use esa definición por defecto únicamente si el usuario no ha definido una función con el mismo nombre en su código C.

```

1  .weak USART1_IRQHandler
2  .thumb_set USART1_IRQHandler, Default_Handler

```

Si el desarrollador escribe en su archivo `main.c`:

```

1  void USART1_IRQHandler(void) {
2      // Código propio para procesar la recepción de datos
3  }

```

El enlazador sustituye automáticamente la dirección del `Default_Handler` por la dirección de esta nueva función dentro de la tabla de vectores, sin necesidad de modificar el archivo de ensamblador original.

4. Interrupciones, sincronización y *callbacks*

Cuando salta una interrupción (cuya dirección está almacenada en la tabla de vectores), el hardware del procesador suspende la ejecución del hilo principal (`main`), guarda automáticamente el contexto (registros R0–R3, R12, LR, PC, xPSR) en la pila y salta a la ISR asignada en la tabla de vectores.

Al compartir variables entre el hilo principal y una rutina de interrupción, surgen problemas de inconsistencia de datos y condiciones de carrera:

- **Uso obligatorio de `volatile`:** Evita que el compilador almacene en registros de la CPU el valor de una variable que puede ser alterada por la ISR en cualquier momento.
- **Operaciones no atómicas:** Si el hilo principal modifica una variable de 32 bits mientras ocurre una interrupción a mitad del proceso de escritura, la ISR leerá datos corruptos.

```
1  #include <stdint.h>
2
3  volatile uint32_t contador_eventos = 0;
4
5  void TIM2_IRQHandler(void) {
6      // esta función se ejecuta automáticamente cuando hay interrupción
7      contador_eventos++;
8  }
9
10 uint32_t leer_contador_seguro(void) {
11     uint32_t copia;
12
13     // Desactivar interrupciones antes de leer
14     __asm volatile ("cpsid i" : : : "memory");
15     copia = contador_eventos; // Lectura protegida
16     // Reactiva interrupciones tras leer
17     __asm volatile ("cpsie i" : : : "memory");
18
19     return copia;
20 }
```

4.1. Barreras de memoria

El código C mantiene un orden lógico. Sin embargo, las optimizaciones del compilador, y el reordenamiento dinámico de instrucciones que sucede a nivel arquitectural, pueden cambiar dicho orden, a fin de mejorar el rendimiento. Esto siempre se hace de tal manera que el resultado final es el mismo, siempre y cuando no haya modificaciones externas de los valores implicados. ¿Y qué pasa con un sistema empotrado con periféricos mapeados en memoria? Pues que puede haber modificaciones externas.

En ciertos momentos, queremos obligar al procesador a completar las operaciones de memoria antes de continuar. Para ello, la arquitectura ARM Cortex-M provee tres instrucciones de barrera de memoria:

- **`__DMB()` (Data Memory Barrier):** Garantiza que todas las accesos a memoria (lecturas y escrituras) previos a la barrera se hayan completado antes de que se inicie cualquier acceso a memoria posterior. No frena la ejecución de instrucciones que no accedan a memoria.

```

1 __attribute__((always_inline)) static inline void __DSB(void) {
2     __asm volatile ("dsb 0xF"::"memory");
3 }

```

- **__DSB() (Data Synchronization Barrier):** Detiene la ejecución de **todas** las instrucciones hasta que las operaciones de acceso a memoria pendientes hayan finalizado.

```

1 __attribute__((always_inline)) static inline void __DMB(void) {
2     __asm volatile ("dmb 0xF"::"memory");
3 }

```

- **__ISB() (Instruction Synchronization Barrier):** Completa todas las instrucciones pendientes del procesador y fuerza a volver a cargar las instrucciones desde memoria o caché. Se utiliza tras modificar la configuración del sistema (como el mapa de memoria o la MPU) para evitar mantener valores antiguos.

```

1 __attribute__((always_inline)) static inline void __ISB(void) {
2     __asm volatile ("isb 0xF"::"memory");
3 }

```

Generalmente, los manuales de uso nos indicarán cuándo es necesario utilizar una de estas barreras al interactuar con los diferentes periféricos y elementos del procesador.

Un caso de uso de `__DSB()` en microcontroladores ocurre al limpiar bits de interrupción en una ISR. Debido al tiempo que tarda el bus en procesar la escritura en el periférico (potencialmente miles de ciclos si el reloj del periférico es muy lento), la CPU podría salir de la ISR antes de que la bandera se limpie físicamente en el registro, re-disparando la misma interrupción.

```

1 void EXTI0_IRQHandler(void) {
2     // ... gestión de ISR
3
4     // ... limpieza de bits
5     EXTI->PR = EXTI_PR_PRO;
6     // ... garantizamos que se haya escrito antes de salir
7     __DSB();
8 }

```

4.2. Punteros a función y Callbacks

Para mantener un diseño de software modular, la capa que interactúa directamente con el hardware (*drivers*) no debe contener lógica de aplicación. El mecanismo estándar para notificar eventos desde dichos *drivers* (o una ISR) hacia la aplicación es la utilización de **punteros a función**. Esta técnica se conoce comúnmente como *callbacks*, ya que la función *hardware* hará una llamada (*callback*) a nuestra función con lógica.

Un puntero a función almacena la dirección de memoria donde está situada, permitiendo su invocación dinámica en tiempo de ejecución.

```

1 #include <stddef.h>
2 #include <stdint.h>
3
4 // Definición del tipo de puntero a función
5 // Será una función que no devuelve nada (void)

```

```

6 // Y que recibe un argumento tipo uint16_t
7 typedef void (*gpio_callback_t)(uint16_t pin);
8
9 // Variable del driver para almacenar el callback
10 static gpio_callback_t exti0_user_callback = NULL;
11
12 // Función de la API pública para registrar el callback
13 // desde el código de aplicación (p.ej: main.c)
14 void gpio_register_callback(gpio_callback_t callback) {
15     exti0_user_callback = callback;
16 }
17
18 // Manejador de la interrupción del hardware
19 void EXTI0_IRQHandler(void) {
20     // ... procesamiento de ISR
21     uint16_t pin = ...;
22
23     // limpieza de bits de interrupción
24     EXTI->PR = EXTI_PR_PRO;
25     __DSB();
26
27     // Llamada al callback
28     if (exti0_user_callback != NULL) {
29         exti0_user_callback(pin);
30         // Ejecuta la función de la capa de aplicación
31         // avisando del estado (en este caso el pin)
32     }
33 }

```